PayPal AI Hackathon: Mandate Gateway Project Plan

Oct 24, 2026 · @MASTER

Hackathon at a glance

The PayPal AI Hackathon has no tracks and no prompts; the only requirement is that a project meaningfully uses both PayPal and AI. Submissions close on Nov 12, 2026, which leaves about 19 days from Oct 24.

Item	Detail
Name	Build What's Next with PayPal and AI (online, worldwide)
Submissions open	Oct 1, 2026
Submissions close	Nov 12, 2026 (some third-party sites say Nov 13; confirm on Devpost)
Winners announced	Dec 21, 2026
Prize pool	$60,000+ cash
Best Overall	1st $12,000, 2nd $8,000, 3rd $5,000
Honorable mentions ($5,000 each)	Most Creative, Most Impactful, Best Demo Delivery, Best Use of PayPal + AI, Best Use of Agentic Commerce
Sponsor bonuses	AG Grid, APIMatic, Bryntum, Channel3, Render
Partners offering free credits	APIMatic, Render, Kernel, Channel3, Postman, Astropods, Elastic, AG Grid, Bryntum

Judging criteria are on the Rules tab of the Devpost page and were not available when this was written; check them before finalizing the pitch.

Ideas considered and scores

Mandate Gateway with the Shopping Brief scores highest at about 9.3 out of 10, and the final plan builds on it. Scores are my own assessment, not a prediction of results.

Idea	Score	One-line verdict
Mandate Gateway + Shopping Brief (improved)	9.3	Safe agent purchasing: policy, product brief, verified agent identity and proof; chosen design
Mandate Gateway + Shopping Brief (first version)	9.0	Fixed the enforcement gap and added the product brief
Mandate / delegated tokens	8.5	Strong problem and PayPal fit, but assumed it could intercept merchant charges
Fraud payment message detection	8.5	Focused and demo-friendly; verifying claims against real account data is the key step
AI escrow for freelancers	7.5	Real impact and PayPal fit; limited by AI judging subjective work and authorization hold limits
SMB procurement negotiator	7.0	Real pain, but crowded and has a trust gap on paying unknown vendors
Checkout negotiation agent	6.5	Easy to build and demo, but weak impact and not novel
Cart recovery negotiator (SMS)	6.5	Same weaknesses, plus messaging consent rules

The negotiation and procurement ideas work best as demo scenes inside the main project, not as the core.

The original Mandate idea and its faults

Mandate was an AI trust layer for payments: users set spending rules in plain English, AI checks each payment, risky ones trigger a plain-language fraud message, and monthly and yearly reports suggest improvements. Its biggest fault was assuming it could sit in the

middle of a payment the agent made on its own.

#	Fault	Severity
1	Likely cannot intercept a merchant's PayPal charge, because merchants capture orders with their own credentials (verify in PayPal's docs)	Critical
2	A rogue or compromised agent can skip it if it has any other way to pay	Critical
3	Natural-language rules are ambiguous and may be judged inconsistently	Critical
4	The AI verifier can be fooled by a well-written scam listing or by prompt injection	Critical
5	Liability is unanswered when it wrongly approves or blocks a payment	Serious
6	False positives annoy users until they loosen or disable limits	Serious
7	Merchants have no way to verify the agent, so they may still block it as a bot	Serious
8	Large companies are building agent payment credentials too	Serious
9	Sandbox data is limited, so reports would run on mock data	Moderate
10	Scope is too large for 19 days (rules, fraud, reports, agent shopping)	Moderate
11	Fraud detection rates cannot be claimed without real data	Moderate
12	Transaction data and messages are sensitive, and a stolen token can spend within its limits	Moderate
13	Real SMS or WhatsApp alerts need approvals and cost money	Moderate
14	Forecasts and improvement reports can be wrong and must not read as financial advice	Moderate

The fix: Mandate Gateway

Mandate Gateway flips the design so the agent cannot pay at all except through it. The agent holds no card or PayPal credentials, only a short-lived token that lets it request purchases.

The flow, in order:

1. **Policy:** the user writes a rule in plain English. AI converts it once into a structured policy (limits, categories, merchants, expiry), the app reads it back in plain words, and the user confirms.
2. **Request:** the agent sends a purchase request with merchant, item, amount and link, plus its Agent Passport.
3. **Check:** plain code checks the request against the policy and the Shopping Brief. AI can add warnings or block, but it can never approve something the code rejects.
4. **Order:** if the request passes, Gateway creates a single-use PayPal order for exactly that amount and merchant, and the user approves it (or it auto-approves inside the policy as a simulated or roadmap feature).
5. **Log:** every decision is written to an audit log, and the user can revoke the agent's token instantly.

The Agent Passport is a signed identity attached to each request: which agent, which human authorized it, the limit and the expiry. A small public page lets merchants verify it, so they can accept trusted agents instead of blocking all bots.

Core rule: the AI can only say no, never yes.

Fault	Solution
Cannot intercept PayPal charges	Gateway is the only place orders are created, so nothing needs intercepting; confirm the Orders and payer approval flow in PayPal's docs
Rogue agent bypasses it	The agent has no payment credentials, only a revocable short-lived token
Ambiguous rules	Parse once, user confirms the structured result, code enforces; merchants use an explicit allowlist or category

Fault	Solution
Verifier fooled or injected	Deterministic checks first; listing text is treated as data, never instructions; AI is advisory and can only tighten
Liability	Human approval over a threshold, full audit log, instant revoke, auto-built dispute pack
False positives	Three tiers (auto-allow, ask, block) that the user can tune
Merchant side missing	Agent Passport with a verification page
Big players in the space	Differentiate on plain-English policy, PayPal-native flow, merchant verification and the AI-can-only-say-no design
Limited sandbox data	Real sandbox orders for the core flow, simulated history for reports, clearly labeled
Unproven detection claims	Build about 50 test scenarios and report the real results
Privacy and security	No inbox access (user pastes or forwards messages), minimal data stored, short-lived tokens
Scope too large	Cut reports and extras; see the build plan

Shopping Brief: what the agent may buy

The Policy controls how much and where the agent can spend; the Shopping Brief controls what it may buy. The user types a request such as "wired gaming mouse, red, with side buttons and a top button, under $40", AI turns it into a structured brief, and the user confirms the readback.

Example brief:

Field	Value	Level
Product type	Mouse	Must
Connection	Wired	Must

Field	Value	Level
Color	Red	Must
Side buttons	Yes	Must
Top button (for example a DPI button above the scroll wheel)	Yes	Must
Max price	$40 (from the Policy)	Must
Use	Gaming	Preference
Rating	4 stars or more	Nice to have
RGB lighting	Yes	Nice to have
Dealbreakers	Wireless-only, refurbished	Must not

Three levels apply: a Must that fails rejects the product, a Nice to have only affects ranking, and a Dealbreaker rejects it instantly.

How matching works:

1. The agent finds candidates (Channel3 offers structured product data, or use a mock catalog).
2. A checker extracts each candidate's specs and compares them to the brief field by field.
3. Each field is marked match, mismatch or unknown, and code computes the score.
4. Gateway creates a PayPal order only if every Must is a match.
5. If a spec is unknown, the agent does not guess; it checks further or asks the user.

The AI reads messy listing text and explains the result, but it never decides a purchase alone. Saved profiles (sizes, brands, dietary needs, recurring items) are optional and cut first if time runs short.

Further improvements

These eight additions raise the project's originality and credibility, and each answers a specific weakness.

Improvement	What it does	Weakness it answers
Plug-in for any agent	Gateway ships as an MCP server and small SDK with a request_purchase call, so any agent can use it	Crowded category; shows it is infrastructure, not one app
Quote-verified matching	Every Must marked a match must quote the exact listing text, and code checks the quote exists; otherwise the result becomes unknown	Hallucinated matches and prompt injection
Proof of Intent	Each purchase stores a signed record linking brief, listing snapshot, policy check and order; a wrong item auto-builds a dispute pack	Liability; PayPal dispute burden
Graduated trust	Starts strict, allows more auto-approvals after clean purchases, tightens after any block or complaint	False positives
Attack My Agent	Built-in red-team suite of about 15 attacks (hidden instructions, price tricks, lookalike products, over-limit retries, fake sellers) with real results shown	Unprovable claims; strong demo moment
Better approvals	One-tap approval page, optional co-signer such as a family member for a parent's agent, always-visible revoke	Trust and usability
Merchant view	Passport verification page and a small view such as 3 verified agent orders and 0 disputes	Merchants blocking agents as bots

Improvement	What it does	Weakness it answers
Scam message checker	The user pastes a suspicious message, and AI checks its claims against real account data (a payment-received email with no matching transaction is flagged)	Fraud message detection idea folded in

Check whether Gateway can sit alongside PayPal's own agent tooling rather than compete with it.

Demo script (about 2.5 minutes)

The demo tells one story: an agent that is safe, accurate and verifiable.

1. Set a plain-English policy and brief, and confirm the readback.
2. Search for the mouse; the rejection table shows a wireless one, a blue one, one with no top button and one with an unknown spec, each with quoted evidence, and one match.
3. The matching mouse goes to PayPal sandbox checkout through Gateway.
4. Press Attack My Agent; blocks and a results summary appear, including a hidden "ignore your rules" listing that fails.
5. Press revoke; the agent's next request is refused.
6. A merchant verifies the Agent Passport on the public page.
7. A wrong item arrives, and the dispute pack is generated automatically.

Build plan (19 days)

The must-haves come first so a working core exists by day 14, with polish and the attack suite after it.

1. **Days 1-3:** check the PayPal sandbox (Orders, payer approval, payment tokens, disputes) and lock the schemas for policy, brief, passport and proof.
2. **Days 4-10:** build policy parsing with the confirm step, brief matching with quote verification, and Gateway with single-use orders.
3. **Days 11-14:** build the Agent Passport page, audit log, instant revoke and Proof of Intent record.
4. **Days 15-17:** build the Attack My Agent suite (about 50 test scenarios in total) and the dispute pack.

5. **Days 18-19:** polish the demo, record it, and write the submission.

Priority	Items
Must have	Policy parsing with confirm step; Gateway with single-use PayPal orders; audit log and revoke; Agent Passport and merchant verify page; brief matching with unknown handling
Should have	Quote-verified evidence; Attack My Agent; scam message checker; dispute pack; one simple monthly summary
Cut unless time allows	Yearly reports, forecasts, saved profiles, real SMS or WhatsApp, PDF export, multi-user accounts

Risks and day-1 checks

The biggest risk is whether the PayPal sandbox supports the order flow Gateway needs; check this on day 1 and mock only what you must, stating it openly in the submission.

Risk	What to do
Sandbox may not support the agent order flow or payer approval you need	Test Orders, payer approval, payment tokens and disputes on day 1; mock restricted parts with realistic data and say so
Scope creep	Hold to the must-have list; cut reports first
Messy real listings with incomplete specs	Use a curated or mock catalog for the demo; keep unknown as its own result
Ambiguous terms such as top button	Ask a short clarifying question
Accuracy claims	Quote only numbers from your own labeled test set
Overlap with big players' agent payment work	Prepare one clear sentence on what is different (plain-English policy, quote-verified matching, passport, proof)
Reports read as financial advice	Label them as insights and suggestions

Risk	What to do
Sensitive data	No inbox access, minimal storage, short-lived tokens, clear consent
Judging criteria unknown	Read the Rules tab on Devpost before finalizing the pitch

Pitch and final score

Mandate Gateway scores about 9.3 out of 10 in my assessment: it fixes the biggest technical flaw, adds a merchant side and a Shopping Brief, and has a visual demo. Feasibility is its lowest part, at about 7 out of 10.

Pitch lines to reuse:

- **One-liner:** a seatbelt for AI shopping agents that checks what they buy, how much they spend and who they are, and proves it afterward.
- **Problem:** people will not give an AI their payment method, and merchants block agents as bots.
- **Design rule:** the AI can only say no; plain code decides what is allowed.
- **For PayPal:** safer agent-initiated purchases, fewer disputes, and a way for merchants to trust agents.
- **Honesty line:** full merchant-side enforcement needs PayPal's support and is the roadmap; today agents must ask Gateway before paying.

What would take it to 9.5: real PayPal sandbox orders end to end, a test set of about 50 scenarios with real pass and fail results, and a tight demo with nothing extra.